BusBidder Spain: Personalized Bus Journey Comparison and Booking System

Kyle Strother

November 3, 2026

Contents

1 Introduction **2**

2 Requirements and Problem Interpretation **2**

3 System Architecture and Software Design **3**

3.1 Data Flow Diagram 3

3.1.1 Trip Data Ingestion and Storage 4

3.1.2 Priority Evaluation and Ranking System 4

3.1.3 User Management and Graphical Interface 4

3.2 Object-Oriented Design Decisions 4

4 Data Management and Validation **5**

5 Searching, Filtering, and Scoring Methodology **6**

5.1 Searching and Filtering Trip data 6

5.2 Weighted Ranking Methodology 6

5.2.1 Code implementation 8

6 Web Interface and Booking Functionality **8**

7 Testing and Experimental Results **8**

8 Traveller Profile Comparisons **9**

9 Critical Evaluation and Limitations **9**

1 Introduction

- Introduce BusBidder Spain and its purpose.
- Explain the problem of comparing competing bus offers.
- Establish the objective: personalized filtering and weighted ranking of journeys.
- Briefly summarize the application's functionality.

2 Requirements and Problem Interpretation

- Identify the main functional requirements.
- Explain key design assumptions and decisions.
- Distinguish mandatory requirements from user preferences.
- Summarize how the implementation addresses the assignment.

3 System Architecture and Software Design

3.1 Data Flow Diagram

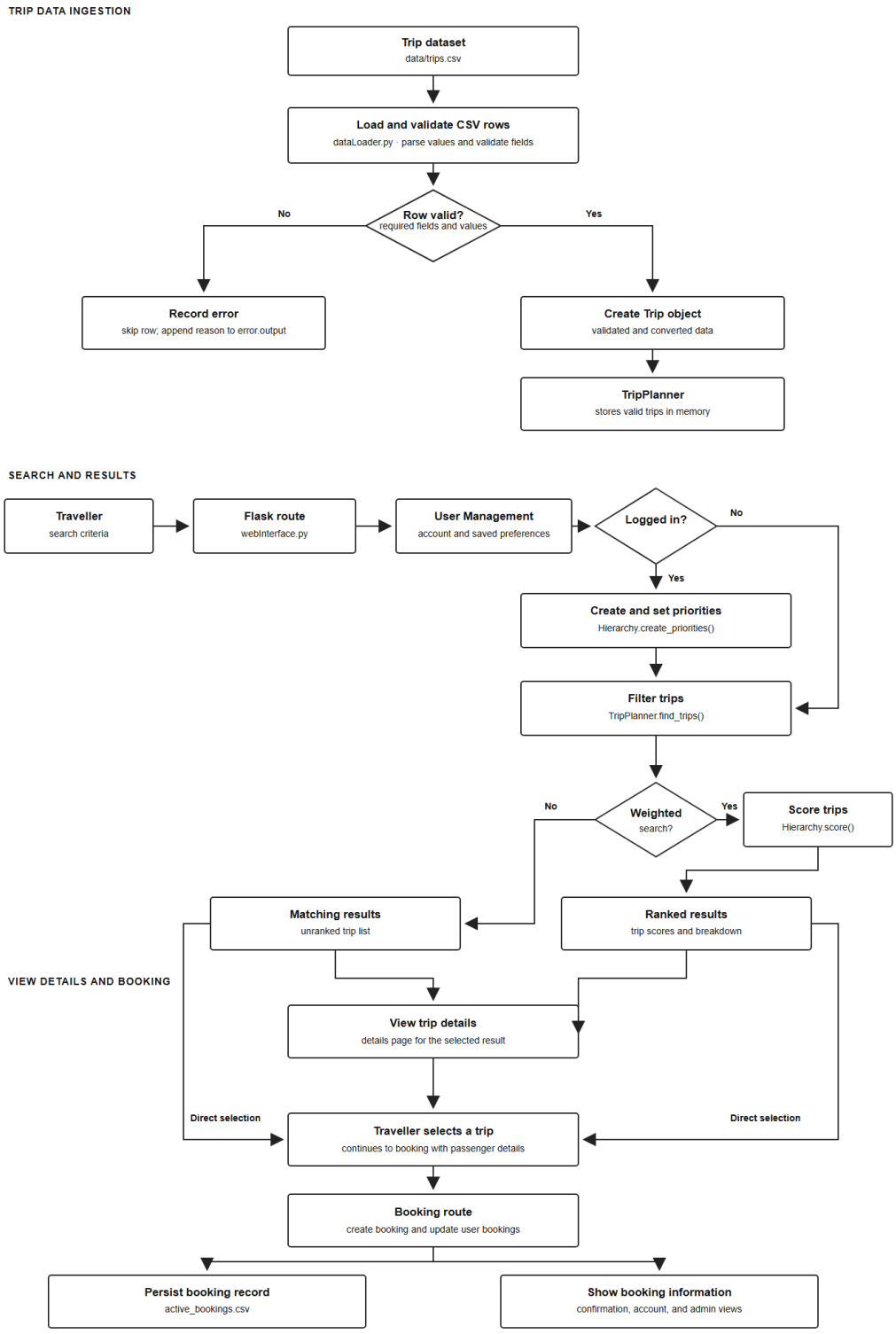


Figure 1: BusBidder data flow from trip ingestion through ranking and booking.

3.1.1 Trip Data Ingestion and Storage

The project structure includes two key modules for reading data and storing it for application processing. The file `dataLoader.py` is responsible for loading the trip data via an CSV data file `data/trips.csv`. The `dataLoader.py` validates it row by row rejecting and reporting failed imports along with reasoning for rejection before they are converted into `Trip` objects. The file `tripsService.py` then manages the storage of `Trip` objects, and then implements the system for querying stored trips, and is also responsible for exporting booking information to external files.

3.1.2 Priority Evaluation and Ranking System

The module `priority.py` is the primary point of determining a scoring of trips. It defines priority types, qualifiers, and evaluation methods and metrics. The module `priorityHierarchy.py` stores and manages the adding and removing of Priority objects and their associated weights, and is how the user initiates a scoring. The file `scoreObject.py` contains the objects used to represent a trip's score and the breakdown of each priority contribution.

3.1.3 User Management and Graphical Interface

The class definitions in `user.py` handle the user account data and model, stores a users weighted priorities; and also establishes a definition of a `Booking` object. The `webInterface.py` Module is the most complex module in the project, and is where the Flask application is defined and run and exports failed data imports and reasoning to the external file `error.output`. It is responsible for exposing the application routes and page rendering logic. It also handles the user input, validating it and querying the data against it. It is through this module that the user can create an account, log in, and save their priority hierarchy, and search for trips have the system display a ranked list of results. The user can then select a trip to book, and the system will save the booking information while updating the Trip data to reduce the number of available seats.

Due to the decision to not include data base logic, the module also handles the persistence of user data and bookings to external files.

3.2 Object-Oriented Design Decisions

The use of separate modules in this design allows the ranking system to operate independently from the web interface, which allowed for better organization and maintainability. A majority of the deeper OOP design decisions were made in the priority evaluation and ranking system, which is the core of the application. Inheritance, composition, and dynamic behaviour are all relevant to the project. Showing it's dynamic nature, a user can maintain a priority hierarchy that changes at runtime and is not fixed once created. Designing priorities in a user specific hierarchy also allows a system to run with simultaneous users and allows each to hold its own preferences.

In designing the priority system, implementing an abstract `EvaluationMethod` to be overwritten by the `Binary` Evaluation class and the `Comparative` Evaluations class within the priority object allows Trip scoring to adapt to different priority types without changing the wider application model.

The application also demonstrates composition through the relationship between a user and their saved priorities, and between a trip and its underlying attributes. The design allows

multiple traveller profiles to be evaluated with the same underlying trip dataset, while giving each weighted priority hierarchy a different result for the same query

4 Data Management and Validation

The dataset for this program is stored in a CSV file, with each row representing a bus trip and its associated attributes. The columns include the following information:

- `offer_id` Required
- `company` Required
- `destination` Required
- `travel_date` Required
- `arrival_date` Required
- `departure_time` Required
- `arrival_time` Required
- `duration_minutes` Required
- `price_euro` Required
- `transfers` Required
- `transfer_via` Optional
- `luggage_allowance_kg` Required
- `changeable` Required
- `refundable` Required
- `available_seats` Required
- `customer_rating` Required
- `rating_scale_max` Required
- `environmental_measure_kg_co2e_per_passenger` Optional

The contents of the supplied dataset are a collection of bus trip offers from various companies, accross different destinations and dates. The given Information on 5 different Sample companies and 15 different valid destinations allowed for the generation of this data.

Each CSV row is checked field by field: Ensures required ID exists, a valid company and destination are given based on the 5 given companies and 15 given destinations, sensible and valid dates and times are given for departure and arrival, valid numeric values and ranges for fields like price, trip duration,number of transfers, luggage allowance, etc.., validates boolean fields are true or false, and a rating that fits its scale. Optional environmental-impact data is parsed when present. If the checks pass, the function converts the values to suitable Python types and creates a Trip object. If any validation fails, the row is rejected and the reason for rejection is logged to an external file `error.output`.

In the provided dataset, there are a total of 125 Trips with 8 Trips being invalid. The 8 invalid trips were rejected due to the following reasons:

- Invalid price data. Price can not be a negative value.
- Invalid trip destination.
- Invalid date given with a month of 13, which is not a valid month.
- Arrival Date occurs before the departure date, which is not a valid trip.
- Invalid customer rating data.
- There needs to be at least 1 available seat for a trip to be valid.
- Price is a mandatory field and cannot be empty.
- A negative number of transfers is impossible for a valid trip

An important note is that the dataset was generated by AI and is not based on real-world data. As such, it may contain unrealistic or inconsistent values but seems to have semi realisitic travel

times, and functions for the purpose of demonstrating the application's functionality.

5 Searching, Filtering, and Scoring Methodology

5.1 Searching and Filtering Trip data

The `tripsService.py` module is responsible for searching and filtering the stored trip data. It provides methods to query trips based on user-defined criteria, such as destination, travel date, departure time, and other attributes. But None of the parameters are required allowing queries to be range from very general to very specific. The filtering process involves iterating through the list of stored trips and applying the specified filters to return a subset of trips that match the user's search options. If no options are given, the system will return all trips in the dataset. The User is able to make these queries through the web interface, where they can select their desired criteria and submit the search. Any specified filters are applied to the trip search and any trip that does not meet the criteria is excluded from the results.

It is in this Search form, that we create "hard constraints" as any trip that doesnt meet a user's supplied constraints should not be displayed to the user. For example if a user is searching to book a trip for the dates 11-04-2027 and 11-08-2027 there is no reason we should be showing them a trip for the 15th. For the implementation of a "soft constraint" we use the weighted ranking system to score and rank trips based on a user's preferences, allowing them to still see non optimal results but display the best options first.

The system setup with allowing a User to have their priorities saved to their account while also allowing them to make rigid searches with hard constraints allows for a flexible and personalized experience. Say a user has a priority set for Lower prices but also knows they cant spend more than 45 Euros, The web form implementation allows them to prevent any trips costing over 45 from being displayed while also displaying trips in a price minded order as well.

5.2 Weighted Ranking Methodology

When a user has a certain idea of the priorities the focus on in a trip are for example Price, Flexibility, Low transfers, short duration, Departure time after 17:00. We can establish a structure that has 100 points and to allow them to rank each priorities importance in respect to a percentage of points. in the following weight:

- Price 37
- Flexibility 26
- Low transfers 19
- Short duration 11
- Departure time after 17:00 7

For this example we can establish 5 test trips as follows:

Company	Destination	Departure time	Duration (min)	Price (EUR)	Transfers	Changeable	Refundable
Camino Express	Barcelona	15:15	399	89.64	0	FALSE	FALSE
CostaLink	Barcelona	13:30	484	60.25	0	FALSE	TRUE
VerdeBus	Barcelona	11:00	513	63.56	0	FALSE	FALSE
NocheVia	Barcelona	20:15	617	53.40	1	TRUE	FALSE
NocheVia	Barcelona	21:15	623	57.22	1	FALSE	TRUE

Table 1: Example bus journeys to Barcelona.

With this hierarchy we see two things: some are measured binarily, and others need a comparison to determine their quality (i.e. what is a lower price in this context).

If we determine average price, and duration of the trips given in an applicable list we can compare the deviation from the average as the percent of its weighted points a priority is awarded The trip with the lowest price should be given 100the same can be said for the duration and number of transfers

The binary priorities are simple, if it meets the criteria we give it whole portion of points so departure after a specific time is yes or no.

Flexibility is slightly more complicated but we can decide to categorize it as 60

With this specific scale and Trip data, we have: - max price of 89.64 and a min of 53.40 - trip duration max: 623 and min: 399 - transfers max: 1 and min: 0

With these scales and specific trip data the design decision is to use the min max normalization formula

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

to determine relative value. The problem here is it is not universal: A low Price is good, but a low customer rating is not. For fields where lower values are better we can take the 1 - the normalization to get an appropriate scale.

Trip 1: - Price:

$$X_{price} = 1 - \left(\frac{89.64 - 53.40}{89.64 - 53.40}\right) = 0$$

so this one gets 0 of 37 points

- Flexibility: This is easy enough it has no refunds and no change so

$$0.6\,(0) + 0.4\,(0) = 0$$

- Transfers: There are none so

$$X_{transfers} = 1 - \left(\frac{0 - 0}{1 - 0}\right) = 1$$

so we get all 19 points here

- Duration: This has the shortest duration so we have

$$X_{duration} = 1 - \left(\frac{399 - 399}{623 - 399}\right) = 1$$

so we get all 11 points here

- Departure time after 17:00: this trip departs at 15:15 so it does not meet this expectation: so it gets 0

Trip 1: point total = 30 / 100 Meaning this is not a great fit.

Doing this for all 5 trips we get the following results:

Company	Price	Flexibility	Transfers	Duration	Departure Time	Total Score
Camino Express	0.00	0.00	19.00	11.00	0.00	30.00
CostaLink	30.01	15.60	19.00	6.83	0.00	71.44
VerdeBus	26.62	0.00	19.00	5.40	0.00	51.02
NocheVia	37.00	10.40	0.00	0.29	7.00	54.69
NocheVia	33.10	15.60	0.00	0.00	7.00	55.70

Table 2: Example bus journeys to Barcelona with their respective scores.

We now have a robust way of scoring and ranking trips based on a user's weighted priorities. The benefit of allowing the user to assign a fully customizable weight is that even if another user ranks those priorities in the same order, if they have slighlty different distributions they will get different score values.

5.2.1 Code implementation

Utilizing the Web form in the `webInterface.py` module, the user can select their priorities and assign weights to them, in an intuitive graphical interface portrayed as a pie chart. One the user submits their preferences, the system will validate the input and create a `PriorityHierarchy` object with the user's priorities and weights. Deeper within the `priority.py` module, each priority is created with a relevant weight and in cases where it is needed a `Qualifer` object is created to add an operator and a value to compare a trips data against to give its scoring a bit more context for what a user values. A good example of the qualifier is the departure time after 17:00 priority, which has a qualifier of operator of "Later than" and value 17:00. so it knows a custom value to score a trip against. Each `Priority` Object has an attribute `PriorityType` which maps it to a given Trip attribute and designates a specific evaluation method. The PriorityType is an Enum that is used to determine which evaluation method to use for a given priority, there are two evaluation methods, Binary and Comparative, and that is used to determine the scoring metric to score Trips.

`Comparative` evaluates attributes such as price relative to the other matching trips, while `Binary` evaluates conditions such as whether a departure meets a specified time preference.

6 Web Interface and Booking Functionality

- Describe the Flask application and its structure.

- Explain the main user-facing functionality.

- Discuss registration, authentication and saved preferences.

- Describe the booking workflow and booking persistence.

- Briefly explain frontend implementation (HTML, CSS, JavaScript).

7 Testing and Experimental Results

- Describe the testing approach and automated test suites.

- Present data validation tests.

- Present scoring and ranking tests.

- Discuss boundary, invalid and missing-data cases.
- Include screenshots and selected application outputs.
- Evaluate whether observed results are reasonable.

8 Traveller Profile Comparisons

- Define three different traveller profiles:

- **Budget-focused traveller**: prioritises low price, low transfer count, and value for money.
- **Business traveller**: prioritises time efficiency, reliability, and preferable departure times.
- **Flexibility-focused tourist**: prioritises refundable tickets, luggage allowance, and adaptable travel arrangements.

- Present their respective priority weights.
- Run all three against the same dataset.
- Compare their resulting rankings and scores.
- Explain why their recommendations differ.

9 Critical Evaluation and Limitations

- Identify the application's main strengths.
- Discuss technical limitations and design trade-offs.
- Evaluate scoring assumptions and normalization.
- Discuss potential unfair outcomes and unintended biases.
- Identify possible improvements and future development.
- Reflect on challenges and lessons learned.